9 Other Shells

While we have spent a great deal of time learning the bash shell, it's not the only "game in town." Unix has had several popular shells and almost all are available for Linux, too. In this adventure, we will look at some of these, mostly for their historical significance. With a couple of possible exceptions, there is very little reason to switch, as bash is a pretty good shell. Some of these alternate shells are still popular on other Unix and Unix-like systems, but are rarely used in Linux except when compatibility with other systems is required.

The Evolution of Shells

The first Unix shell was developed in 1971 by Ken Thompson who, along with Dennis Richie, created Unix at AT&T Bell Telephone Laboratories. The *Thompson shell* introduced many of the core ideas that we see in shells today. These include I/O redirection, pipelines, and the ability to place processes in the background. This early shell was intended only for interactive use, not for use as a programming language.

The Thompson shell was followed in 1975 by the *Mashey shell*, written by John Mashey. This shell extended the Thompson shell to support shell scripting by including variables, a built-in if/then/else, and other rudimentary flow control constructs.

At this point we come to a big split in shell design philosophies. In 1978 Steve Bourne created the *Bourne shell*. The following year, Bill Joy (the original author of `vi`) released the *C shell*.

The Bourne shell added a lot of features that greatly improved shell scripting. These included flow control structures, better variables, command substitutions, and here scripts. The Bourne shell contains much of the functionality that we see in the bash shell today.

On the other hand, the C shell was designed to improve interactive use by adding command history and job control. The C shell, as its name would imply, uses a syntax that mimics the C programming language. C language programmers abounded in the Unix community, so many preferred this style. Ironically, the C shell is not very good at scripting. For example, it lacks user defined functions and the shell's parser (the portion of the shell that reads and figures out what the script is saying) suffers from serious limitations.

In 1983, in an effort to improve the Bourne shell, David Korn released the *Korn shell*. Command history, job control, associative arrays, vi and Emacs style command editing are among the features that were added. In the 1993 release (known as *ksh93*), floating point arithmetic was added. The Korn shell was good for both interactive use and

scripting. Unfortunately, the Korn shell was proprietary software distributed under license from AT&T. This changed in 2000 when it was released under an open source license.

When POSIX standardized the shell for use on Unix systems, it specified a subset of the Korn shell that would be largely compatible with the earlier Bourne shell. As a result, most Bourne-type shells now conform to the POSIX standard, but include various extensions.

Partially in response to the proprietary licensing of the Korn shell, the GNU project developed `bash`, which includes many Korn shell features. The first version, written by Brian Fox was released in 1989 and is today maintained by Chet Ramey. Bash is best known as the default shell in most Linux distributions. It is also the default shell in versions of macOS; however, due to Apple's obsession with secrecy and lock-down, they refuse to update `bash` to version 4 because of provisions in the GNU GPLv3.

Since the development of `bash`, one new shell has emerged that is gaining traction among Linux and MacOS users. It's the *Z shell* (zsh). Sometimes described as "the Emacs of shells" because of its large feature set, `zsh` adds a number of features to enhance interactive use.

Modern Implementations

Modern Linux users have a variety of shell programs from which to choose. Of course, the overwhelming favorite is `bash`, since it is the default shell supplied with most Linux distributions. That said, users migrating from other Unix and Unix-like systems may be more comfortable with other shells. There is also the issue of portability. If a script is required to run on multiple Unix-like systems, then care must be taken to either: 1) make sure that all the systems are running the same shell program, or 2) write a script that conforms to the POSIX standard, since most modern Bourne shell derivatives are POSIX complaint.

A Reference Script

In order to compare the various shell dialects, we'll start with this `bash` script taken from Chapter 33 of TLCL:

```
#!/bin/bash

# longest-word : find longest string in a file

for i; do
  if [[ -r "$i" ]]; then
    max_word=
    max_len=0
    for j in $(strings "$i"); do
      len=${#j}
      if (( len > max_len )); then
        max_len=$len
```

```
        max_word=$j
      fi
    done
    echo "$i: '$max_word' ($max_len characters)"
  fi
done
```

dash - Debian Almquist Shell

The Debian Almquist shell is Debian's adaptation of the *Almquist shell* (ash) originally written in the 1980s by Kenneth Almquist. The ash shell is the default shell on several of the BSD flavors of Unix. `dash`, like its ancestor `ash`, has the advantage of being small and fast; however, it achieves this by forgoing conveniences intended for interactive use such as command history and editing. It also lacks some builtin commands, relying instead on external programs. Its main use is the execution of shell scripts, particularly during system startup. On Debian and related distributions such as Ubuntu, `dash` is linked to `/bin/sh`, the shell used to run the system initialization scripts.

`dash` is a POSIX compliant shell, so it supports Bourne shell syntax with a few additional Korn shell features:

```
#!/bin/dash

# longest-word.dash : find longest string in a file

for i; do
  if [ -r "$i" ]; then
    max_word=
    max_len=0
    for j in $(strings "$i"); do
      len=${#j}
      if [ $len -gt $max_len ]; then
        max_len=$len
        max_word=$j
      fi
    done
    echo "$i: '$max_word' ($max_len characters)"
  fi
done
```

Here we see that the dash script is mostly the same as the `bash` reference script, but we do see some differences. For one thing, `dash` does not support the '[[' syntax for conditional tests; it uses the older Bourne shell syntax. The POSIX specification is also missing the *((expression))* syntax for arithmetic expansion, nor does it support brace expansion. `dash` and the POSIX specification do support the *$(cmd)* syntax for command substitution in addition to the older `cmd` syntax.

tcsh - TENEX C Shell

The `tcsh` program was developed in the early 1980s by Ken Greer as an enhanced replacement for the original `csh` program. The name TENEX comes from the operating system of the same name, which was influential in the design of the interactive features in

`tcsh`. Compared to `csh`, `tcsh` added additional command history features, Emacs and vi-style command line editing, spelling correction, and other improvements intended for interactive use. Early versions of Apple's OS X used `tcsh` as the default shell. It is still the default root shell on several BSD distributions.

`tcsh`, like the C shell, is not POSIX compliant as we can see here:

```
#!/usr/bin/tcsh

# longest-word.tcsh : find longest string in a file

foreach i ($argv)
  set max_word=""
  set max_len=0
  foreach j (`strings $i`)
    set len=$%j
    if ($len > $max_len) then
      set max_word=$j
      set max_len=$len
    endif
  end
  echo "$1 : $max_word ($max_len characters)"
end
```

Our `tcsh` version of the script demonstrates many differences from Bourne style syntax. In C shell, most of the flow control statements are different. We see for example, that the outer loop starts with a `foreach` statement incrementing the variable `i` with succeeding values from the word list `$argv`. `argv`, taken from the C programming language, refers to an array containing the list of command line arguments.

While this simple script works, `tcsh` is not very capable when things get more complicated. It has two major weaknesses. First, it does not support user-defined functions. As a workaround, separate scripts can be called from the main script to carry out the individual functions. Second, many complex constructs easily accomplished with the POSIX shell, such as:

```
{ if [[ "$a" ]]; then
    grep "string1"
  else
    grep "string2"
  fi
} < file.txt
```

are not possible because the C shell parser cannot handle redirection with flow control statements. The parser also makes quoting very troublesome.

ksh - Korn Shell

The Korn shell comes in several different flavors. Basically, there are two groups, ksh88 and ksh93, reflecting the year of their release. There is a public domain version of ksh88 called `pdksh`, and more official versions of both `ksh88` and `ksh93`. All three are available for Linux. `ksh93` would be the preferred version for most users, as it is the version found

on most modern commercial Unix systems. During installation is it often symlinked to `ksh`.

```
#!/usr/bin/ksh

# longest-word.ksh : find longest string in a file

for i; do
  if [[ -r "$i" ]]; then
    max_word=
    max_len=0
    for j in $(strings "$i"); do
      len=${#j}
      if (( len > max_len )); then
        max_len=$len
        max_word=$j
      fi
    done
    print "$i: '$max_word' ($max_len characters)"
  fi
done
```

As we can see in this example, `ksh` syntax is very close to `bash`. The one visible difference is the `print` command used in place of `echo`. Korn shell has `echo` too, but `print` is the preferred Korn shell command for outputting text. Another subtle difference is the way that pipelines work in `ksh`. As we learned in Chapter 28 of TLCL, a construct such as:

```
#!/bin/bash
str=""
echo "foo" | read str
echo $str
```

always produces an empty result because, in `bash` pipelines, each command in a pipeline is executed in a subshell, so its data is destroyed when the subshell exits. In this example, the final command (`read`) is in a subshell, and thus `str` remains empty in the parent process.

In `ksh`, the internal organization of pipelines is different. When we do this in `ksh`:

```
#!/usr/bin/ksh
str=""
echo "foo" | read str
echo $str
```

The output is “foo” because in the `ksh` pipeline, the `echo` is in the subshell rather than the `read`.

zsh - Z Shell

At first glance, the Z shell does not differ very much from `bash` when it comes to scripting:

```
#!/bin/zsh

# longest-word.zsh : find longest string in a file
```

```
for i; do
  if [[ -r "$i" ]]; then
    max_word=
    max_len=0
    for j in $(strings "$i"); do
      len=${#j}
      if (( len > max_len )); then
        max_len=$len
        max_word=$j
      fi
    done
    print "$i: '$max_word' ($max_len characters)"
  fi
done
```

It runs scripts the same way that `bash` does. This is to be expected, as `zsh` is intended to be a drop-in replacement for `bash` in most cases. A couple of things to note however. First, `zsh` handles pipelines like the Korn shell does; the last command in a pipeline is executed in the current shell. Second, in `zsh`, the first element of an array is index 1, not 0 as it in `bash` and `ksh`.

Where `zsh` does differ significantly is in the number of bells and whistles it provides for interactive use (some of which can be applied to scripting as well). Let's take a look at a few:

Tab Completion

Many kinds of tab completion are supported by `zsh`. These include command names, command options, and arguments.

When using the `cd` command, repeatedly pressing the tab key first displays a list of the available directories, then begins to cycle through them. For example:

```
me@linuxbox ~ $ cd <tab>

me@linuxbox ~ $ cd <tab>
Desktop/    Documents/  Downloads/  Music/  Pictures/   Public/
Templates/  Videos/

me@linuxbox ~ $ cd Desktop/<tab>
Desktop/    Documents/  Downloads/  Music/  Pictures/   Public/
Templates/  Videos/

me@linuxbox ~ $ cd Documents/
Desktop/    Documents/  Downloads/  Music/  Pictures/   Public/
Templates/  Videos/
```

`zsh` can be configured to display a highlighted selector on the list of directories, and we can use the arrow keys to directly move the highlight to the desired entry in the list to select it.

We can also switch directories by replacing one part of a path name with another:

```
me@linuxbox ~ $ cd /usr/local/share
```

```
me@linuxbox share $ cd share bin
me@linuxbox bin $ pwd
/usr/local/bin
```

Pathnames can be abbreviated as long as they are unambiguous. If we type:

```
me@linuxbox ~ $ ls /u/l/share<tab>
```

`zsh` will expand it into:

```
me@linuxbox ~ $ ls /usr/local/share/
```

That can save a lot of typing!

Help for options and arguments is provided for many commands. To invoke this feature, we type the command and the leading dash for an option, then hit the tab key:

```
me@linuxbox ~ $ rm -<tab>
--force             -f      -- ignore nonexistent files, never prompt
--help                      -- display help message and exit
-i                          -- prompt before every removal
-I                          -- prompt when removing many files
--interactive               -- prompt under given condition
                               (defaulting to always)
--no-preserve-root          -- do not treat / specially
--one-file-system           -- stay within file systems of files given
                               as arguments
--preserve-root             -- do not remove / (default)
--recursive         -R  -r  -- remove directories and their contents
                               recursively
--verbose           -v      -- explain what is being done
--version                   -- output version information and exit
```

This displays a list of options for the command, and like the `cd` command, repeatedly pressing `tab` causes `zsh` to cycle through the available options.

Pathname Expansion

The Z shell provides several powerful additions to pathname expansion that can save steps when specifying files as command arguments.

We can use "**" to cause recursive expansion. For example, if we wanted to list every file name ending with `.txt` in our home directory and its subdirectories, we would have to do this in `bash`:

```
me@linuxbox ~ $ find . -name "*.txt" | sort
```

In `zsh`, we could do this:

```
me@linuxbox ~ $ ls **/*.txt
```

and get the same result.

And if that weren't cool enough, we can also add *qualifiers* to the wildcard to perform many of the same tests as the `find` command. For example:

```
me@linuxbox ~ $ **/*.txt(@)
```

will only display the files whose names end in `.txt` and are symbolic links.

There are many supported qualifiers and they may be combined to perform very fine grained file selection. Here are some examples:

Qualifier	Description	Example
.	Regular files	`ls *.txt(.)`
/	Directories	`ls *.txt(/)`
@	Symbolic links	`ls *.txt(@)`
*	Executable files	`ls *(*)`
F	Non-empty (“full”) directories	`ls *(F)`
/^F	Empty directories	`ls *(/^F)`
m*n*	Modified exactly *n* days ago	`ls *(m5)`
m-*n*	Modified less than *n* days ago	`ls *(m-5)`
m+*n*	Modified more than *n* days ago	`ls *(m+5)`
L0	Empty (zero length) file	`ls *(L0)`
LM+*n*	File larger than *n* megabytes	`ls *(LM+5)`
LK-*n*	File smaller than *n* kilobytes	`ls *(LK-100)`

Z shell pathname expansions

Global Aliases

Z shell provides more powerful aliases. With `zsh` we can define an alias in the usual way, such as:

```
me@linuxbox ~ $ alias vi='/usr/bin/vim'
```

and it will behave just as it would in `bash`. But we can also define a *global alias* that can be used at any position on the command line, not just at the beginning. For example, we can define a commonly used file name as an alias:

```
me@linuxbox ~ $ alias -g LOG='/var/log/syslog'
```

and then use it anywhere on a command line:

```
me@linuxbox ~ $ less LOG
```

The use of an uppercase alias name is not a requirement, it’s just a custom to make its use easier to see. We can also use global aliases to define common redirections:

```
me@linuxbox ~ $ alias -g L='| less"
```

or

```
me@linuxbox ~ $ alias -g W='| wc -l'
```

Then we can do things like this:

```
me@linuxbox ~ $ cat LOG W
```

to display the number of lines in `/var/log/syslog`.

Suffix Aliases

What's more, we can define aliases to act like an "open with…" by defining a *suffix alias*. For example, we can define an alias that says all files that end with ".txt" should be viewed with less:

```
me@linuxbox ~ $ alias -s txt='less'
```

Then we can just type the name of a text file, and it will be opened by the application specified by the alias:

```
me@linuxbox ~ $ dir-list.txt
```

How cool is that?

Improved History Search

`zsh` adds a neat trick to history searching. In `bash` (and `zsh` too) we can perform a reverse incremental history search by typing `Ctrl-r`, and each subsequent keystroke will refine the search. `zsh` goes one better by allowing us to simply type a few letters of the desired search string on the command line and then press up-arrow. It moves back through the history to find the first match, and each time we press the up-arrow, the next match is displayed.

Environment Variable Editing

`zsh` provides a shell builtin called `vared` for editing shell variables. For example, if we wanted to make a quick change to our `PATH` variable we can do this:

```
me@linuxbox ~ $ vared PATH
```

and the contents of the `PATH` variable appear in the command editor, so we can make a change and press Enter and the change takes effect.

Frameworks

We have only touched on a few of the features available in `zsh`. It has a lot. But with a large feature set comes complexity, and configuring `zsh` to take advantage of its full potential can be daunting. Heck, its man page is a only a table of contents to the other 10+ man pages that cover various topics. Fortunately, communities have sprung up to provide *frameworks* that supply ready-to-use configurations and add-ons for `zsh`. By far, the most popular of these is Oh-My-Zsh, a project led by Robby Russell.

Oh-My-Zsh is a large collection of configuration files, plugins, aliases, and themes. It offers support for tailoring `zsh` for many types of common tasks, particularly software development and system administration.

Changing to Another Shell

Now that we have learned a little about the different shells available for Linux, how can we experiment with them? First, we can simply enter the name of the shell from our `bash` prompt. This will launch the second shell as a child process of `bash`:

```
me@linuxbox ~ $ tcsh
%
```

Here we have launched `tcsh` from the `bash` prompt and are presented with the default `tcsh` prompt, a percent sign. Since we have not yet created any startup files for the new shell, we get a very bare-bones environment. Each shell has its own configuration file(s) for interactive use just as `bash` has the `.bashrc` file to configure its interactive sessions.

Here is a table that lists the configuration files for each of the shells when used as an interactive (i.e., not a login) shell:

Shell	**Configuration File(s)**
`dash`	User-defined by setting the ENV variable in `~/.profile`
`bash`	`~/.bashrc`
`ksh`	`~/.kshrc`
`tcsh`	`~/.tchrc`
`zsh`	`~/.zshrc`

Interactive shell configuration files

We'll need to consult the respective shell's man page (always a fun exercise!) to see the complete list of shell features. Most shells also include additional documentation and example configuration files in the `/usr/share/doc` directory.

To exit our temporary shell, we simply enter the `exit` command:

```
% exit
me@linuxbox ~ $
```

Once we are done with our experimentation and configuration, we can change our default shell from `bash` to our new shell by using the `chsh` command. For example, to change from `bash` to `zsh`, we could do this:

```
me@linuxbox ~ $ chsh
password:
Changing the login shell for me
Enter the new value, or press ENTER for the default
   Login Shell [/bin/bash]: /usr/bin/zsh

~ 23:30:40
$
```

We are prompted for our password and then prompted for the name of the new shell whose name must appear in the `/etc/shells` file. This is a safety precaution to prevent an invalid name from being specified and thus preventing us from logging in again. That would be bad.

Summing Up

Because of the growing popularity of Linux among Unix-like operating systems, `bash` has become the world's predominant shell program. It has many of the best features of earlier shells and a few tricks of its own. However, if light weight and quick script execution is needed (for example, in embedded systems), `dash` is a good choice. Likewise, if working with other Unix systems is required, `ksh` or `tcsh` will provide the necessary compatibility. For the adventuresome among us, the advanced interactive features of `zsh` can enhance our day-to-day shell experience.

Further Reading

Shells and their history:

- A history of Unix shells from IBM Developer Works: https://developer.ibm.com/tutorials/l-linux-shells/

C shell:

- A comparison of bash and tcsh syntax by Joe Linoff: http://joelinoff.com/blog/?page_id=235
- Tom Christiansen's famous "Csh Programming Considered Harmful" explains the many ways that csh bugs out when scripting: https://www-uxsup.csx.cam.ac.uk/misc/csh.html
- And on a related note, here are the "Top Ten Reasons not to use the C shell" by Bruce Barnett: https://www.grymoire.com/unix/CshTop10.txt

Korn shell:

- Korn shell documentation: http://www.kornshell.com/doc/
- The on-line version of "Learning the Korn Shell" from O'Reilly: http://web.deu.edu.tr/doc/oreily/unix/ksh/index.htm

Z shell:

- Brendon Rapp's slide presentation on "Why zsh Is Cooler Than Your Shell": https://www.slideshare.net/jaguardesignstudio/why-zsh-is-cooler-than-your-shell-16194692
- Joe Wright's list of favorite zsh features: https://code.joejag.com/2014/why-zsh.html
- David Fendrich's "No, Really. Use Zsh.": http://fendrich.se/blog/2012/09/28/no/

- Nacho Caballero's "Master Your Z Shell with These Outrageously Useful Tips": http://reasoniamhere.com/2014/01/11/outrageously-useful-tips-to-master-your-z-shell/
- Home page for Oh-My-Zsh: https://ohmyz.sh/